

Procesamiento de Datos Masivos

Clase 07 - Map-Reduce y Apache Spark

Qué haremos hoy

- Repaso de la clase pasada
- PageRank: teoría de funcionamiento e implementación
- Otras aplicaciones de MapReduce/PySpark

Repaso MapReduce y PySpark





Idea general

- Aprovechar cómputo distribuido para procesar grandes volúmenes de datos en paralelo
- Frameworks nos ayudarán para abstraer comunicación y resolver fallos en procesamiento
- Nodo manager, workers mapper y reducers, shuffle

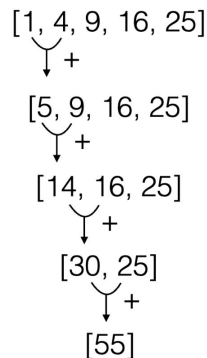
MapReduce

- Función de mapeo: Aplica misma función a todas las filas del dataset.
- Función de reducción: Su objetivo es pasar de un iterable a un elemento
- Trabaja leyendo y escribiendo en disco

```
numeros = [1, 2, 3, 4, 5]
cuadrados = map(lambda x: x ** 2, numeros)

def suma(x, y):
    return x + y

total = functools.reduce(suma, numeros)
```



(Py)Spark

- Extiende MapReduce
- Operadores de más alto nivel
- Trabaja en memoria, siendo mucho más eficiente
- Lazy evaluation. Spark construye un DAG, no ejecuta línea por línea

PySpark (clase II)





Malas prácticas

- Uso de **collect** sin razón
 - Cuello de botella
 - Ejecuta todo el DAG completo
 - Se lleva todo al nodo driver que no estpa hecho para grandes volúmenes
- Recomputar innecesariamente. Spark no guarda resultados intermedios a menos que le digamos



```
df_filtered = df.filter(df.amount > 1000)

df_filtered.count()
df_filtered.collect() # recalcula TODO
```



```
df_filtered = df.filter(df.amount > 1000).cache()

df_filtered.count()
df_filtered.collect()
```



Malas prácticas

- Uso de **collect** sin razón
 - Cuello de botella
 - Ejecuta todo el DAG completo
 - Se lleva todo al nodo driver que no estpa hecho para grandes volúmenes
- Recomputar innecesariamente. Spark no guarda resultados intermedios a menos que le digamos
- Mal particionamiento. Usar pocas (*underutilization*) o muchas (*overhead*) particiones

Buenas prácticas

- Filtrar antes de operaciones que hagan shuffle

- Correcto uso de JOINS

```
from pyspark.sql.functions import broadcast  
big_df.join(broadcast(small_df), "id")
```

- Cachear resultados

Algunos operadores



- `rdd.flatMap(lambda x: x.split(" "))`
- `rdd.filter(lambda x: x > 10)`
- `rdd.reduceByKey(lambda a, b: a + b)`
- Operadores con shuffle (los caros):
 - `groupBy`
 - `join`
 - `distinct`
- `orderBy`



Aplicaciones: PageRank





Aplicaciones



- Ingeniería de datos: Uso en ETL/ELT's
- Analítica a gran escala
- Minería de datos: clasificación y detección de patrones
- Procesamiento de grafos
- ...



PageRank

Consulta: Mejor Universidad

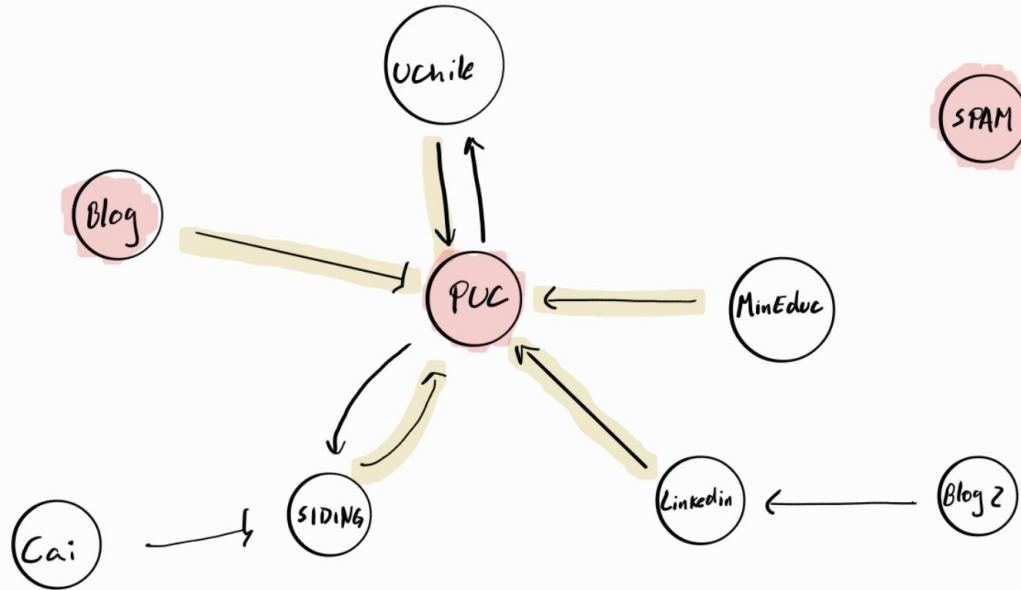
SPAM.cl
Universidad univer-
sidad universidad
universidad univer...

Universidad
• Investigaciones
• Académicos
• ...

Blog
Mis universidades
favoritas

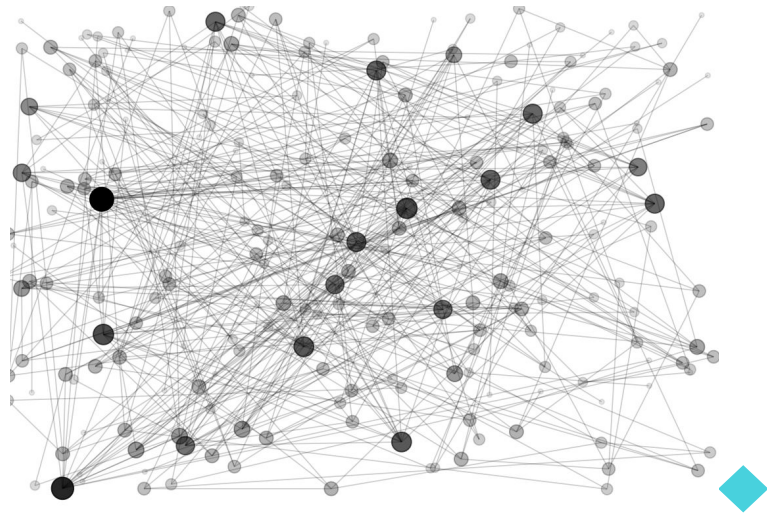
PageRank

Consulta: Mejor Universidad



PageRank

La idea es simular un navegante de la web aleatorio, que hace clicks y va cambiando de páginas



Queremos calcular en qué páginas **es más probable** que termine

El algoritmo incluye un *damping factor* d , que corresponde a la probabilidad que deje de hacer links y salte a una página aleatoriamente

PageRank

Input: Un grafo

Output: Distribución de probabilidades de que un usuario termine en una cierta página

Parámetros: *damping factor* d

Consideraciones: Ignoraremos los self-loops y múltiples relaciones $A \rightarrow B$ se interpretan solo como una

PageRank

- t_0 : Todos los nodos (páginas) tienen la misma probabilidad
- t_i : Cada página le entrega parte de su probabilidad (alcanzada en t_{i-1}) a todas aquellas a las que tiene un link
- En cada iteración, la probabilidad de un nuevo salto se atenúa según el factor d
- Terminamos cuando la diferencia entre iteraciones este bajo un delta

PageRank

En la iteración 0 definimos el PageRank para cada nodo como:

$$PR_0(n_i) = 1/N$$

Y para las siguientes iteraciones como:

$$PR_t(n_i) = \frac{1-d}{N} + d \sum_{n_j \in In(n_i)} \frac{PR_{t-1}(n_j)}{Out(n_j)}$$

PageRank

$$PR_t(n_i) = \frac{1 - d}{N} + d \sum_{n_j \in In(n_i)} \frac{PR_{t-1}(n_j)}{Out(n_j)}$$

Diagram illustrating the PageRank formula and its components:

- $PR_t(n_i)$: PageRank del nodo i en la iteración t
- N : Número de nodos en el grafo
- d : Damping factor, usualmente 0.85
- $\sum_{n_j \in In(n_i)}$: Suma sobre los nodos n_j que tienen una arista entrando al nodo i
- $PR_{t-1}(n_j)$: PageRank del nodo j en la iteración $t-1$
- $Out(n_j)$: Número de aristas saliendo del nodo j

PageRank

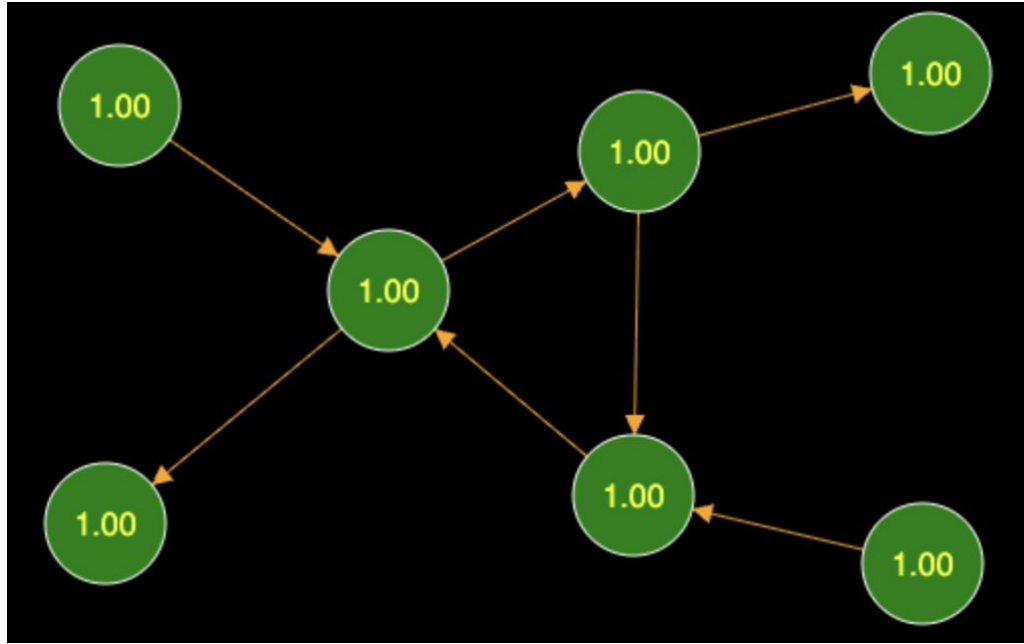
En cada iteración calculamos el siguiente valor:

$$\sqrt{(PR_t(nodo_1) - PR_{t-1}(nodo_1))^2 + \dots + (PR_t(nodo_N) - PR_{t-1}(nodo_N))^2}$$

Hasta que tome un valor "suficientemente pequeño"

PageRank

T0

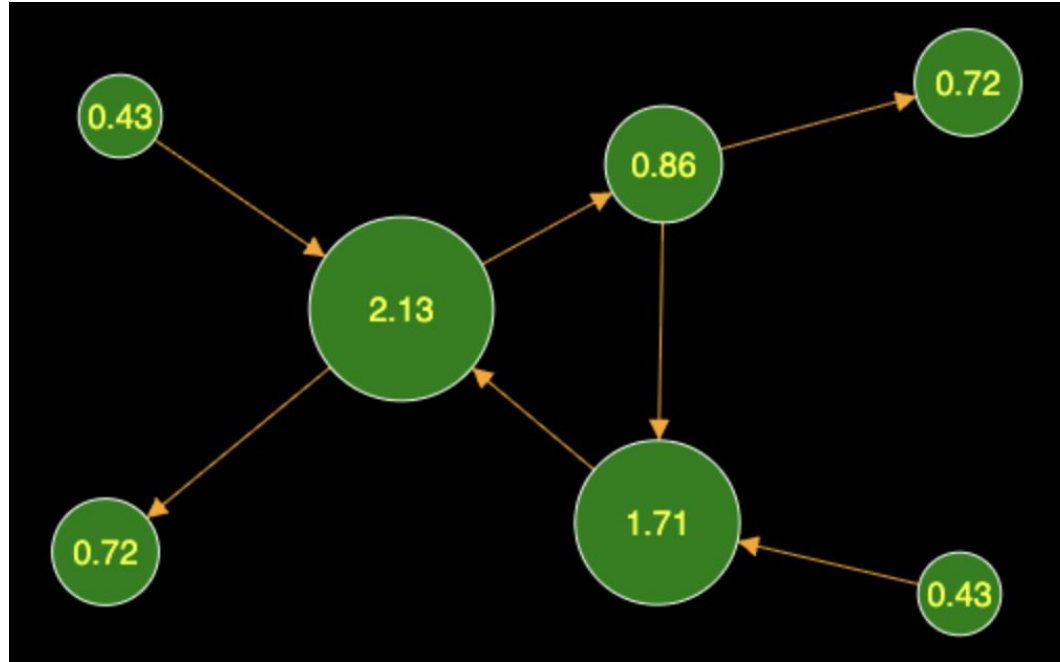


<https://www.mathsuniverse.com/pagerank>

**El cálculo no lo hace igual a la fórmula de antes, pero la idea es la misma

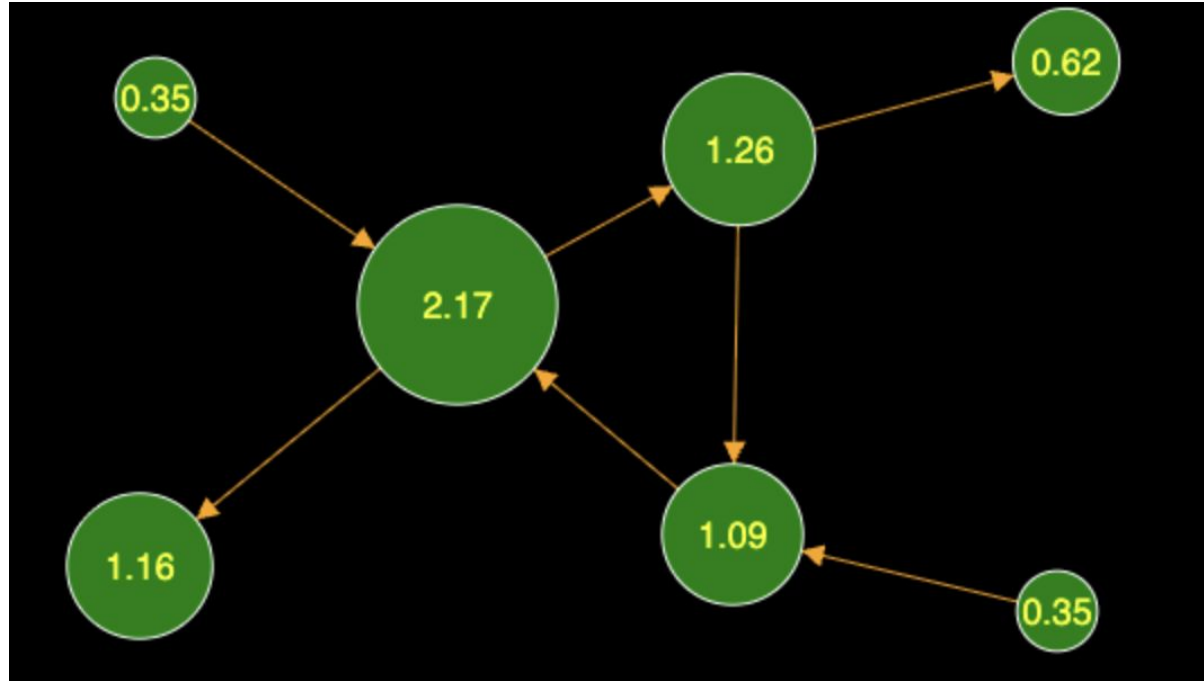
PageRank

T1



PageRank

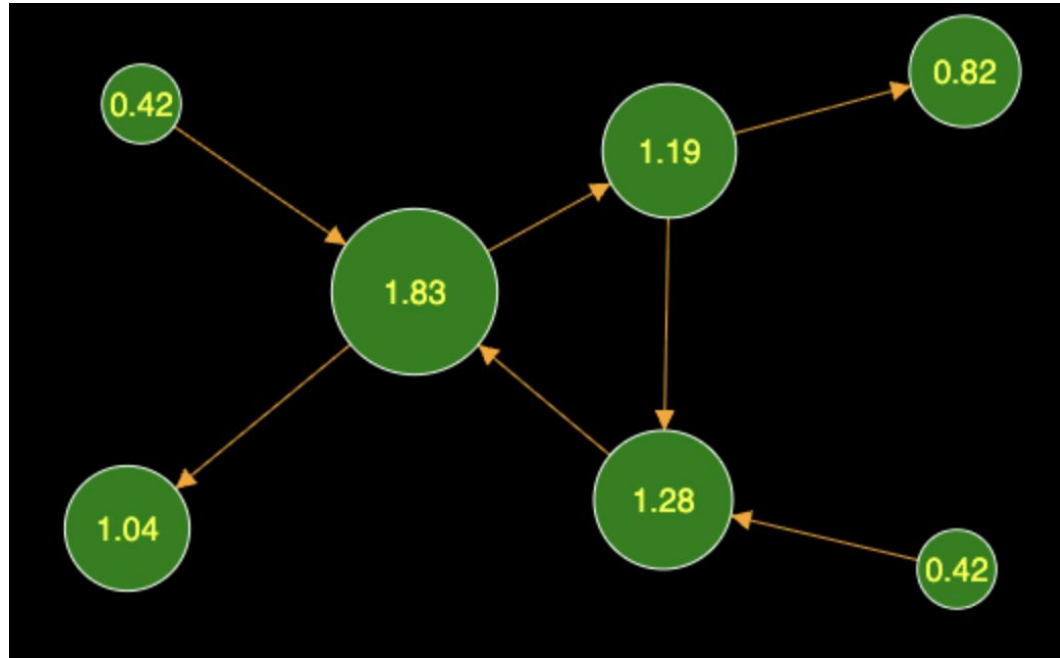
T2



PageRank

T7 en adelante:

Converge

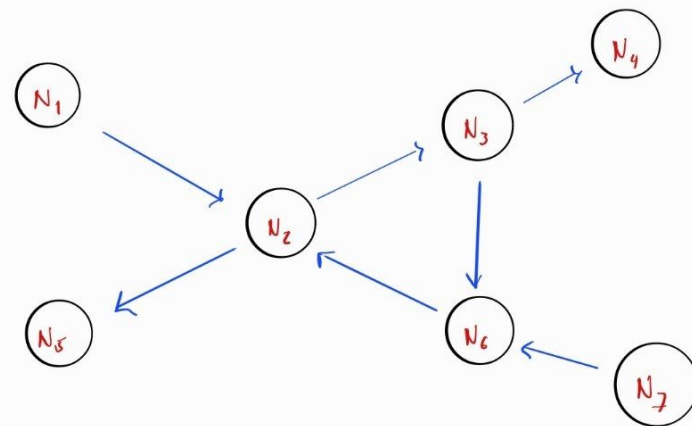


PageRank

- Si d es muy grande, necesitamos muchas iteraciones para converger
- Si d es muy chico, todo converge a 1

PageRank - implementación

- Tendremos links y ranks



$links = \{$
 $N_1 : [N_2],$
 $N_2 : [N_3, N_5],$
 $N_3 : [N_4, N_6],$
 $\dots$
 $\}$

$ranks = \{$
 $N_1 = 1/2,$
 $N_2 = 1/2,$
 $\dots$
 $\}$

PageRank - implementación

- Tendremos links y ranks
- A cada nodo, obtendremos el ranking que sale y a quién va

Join links y ranks:

$$\text{joined} = \left\{ \begin{array}{l} N_1 : \left(\frac{1}{2}, [N_2] \right), \\ N_2 : \left(\frac{1}{2}, [N_3, N_5] \right), \\ N_3 : \left(\frac{1}{2}, [N_4, N_6] \right), \\ \dots \end{array} \right\}$$

Mapeamos

$$\text{ranks} = \text{lambda row : } \left(\text{in_node}, \frac{\text{rank}}{\| \text{relations} \|} \right) \text{ for rel in relations}$$

$$\Rightarrow \left[(\text{node}_i, \text{rank}), (\text{node}_i, \text{rank}'), (\text{node}_j, \text{rank}'), \dots \right]$$

PageRank - implementación

- Tendremos links y **ranks**
- A cada nodo, obtendremos el ranking que sale y a quién va
- Reagrupamos por llave, obteniendo la suma recibida
- Aplicamos ponderador

$ranks = \text{reduce By Key (sum)}$

$ranks = \text{lambda row: } \frac{0.15}{N} + 0.95 \cdot row$

- ◆ • Tenemos nuevos **ranks** con los que volver a iterar

PageRank - implementación



```
for i in range(iteraciones):  
    # 1. join (tener links + rank)  
    joined = links.join(ranks)  
  
    # 2. map: repartir rank  
    contribs = joined.flatMap(lambda (node, (neighbors, rank)):  
        [(n, rank / len(neighbors)) for n in neighbors])  
  
    # 3. reduce: sumar contribuciones  
    new_ranks = contribs.reduceByKey(sum)  
  
    # 4. aplicar damping  
    ranks = new_ranks.mapValues(lambda r: 0.15 + 0.85 * r)
```



PageRank

- PageRank no necesitamos conocer el estado del grafo completo para cada nodo
- Nos basta con tener información de vecinos
- Aprovechar esto, nos ayuda a poder computar en paralelo sin la necesidad de supercomputadores

Aplicaciones: Multiplicación de matrices

Multiplicación de matrices

- Sean las matrices $A \in \mathbb{R}^{m \times n}$ $B \in \mathbb{R}^{n \times p}$

- Queremos calcular $C = A \times B \in \mathbb{R}^{m \times p}$

- Complejidad: $O(m \cdot n \cdot p)$

- Si consideramos matriz cuadrada de $n=10^6 \rightarrow$ costo de 10^{18}

- No cabe en memoria y muy costoso de operar

$$C_{i,j} = \sum_{k=1}^n A_{i,k} \cdot B_{k,j}$$

Estamos usando algoritmo base, pero mejor estrategia no es mucho mejor: $\approx O(n^{2.807})$

Multiplicación de matrices

- Volvamos a ver fórmula:
$$C_{i,j} = \sum_k A_{i,k} \cdot B_{k,j}$$
- Cada operación es independiente del resto
- Lógica:
 - Emparejar filas y columnas correspondientes
 - Multiplicar
 - Sumar



Vamos a la pizarra

Multiplicación de matrices

- En realidad no ganamos mucho en matrices densas: $O\left(\frac{m \cdot n \cdot p}{N}\right)$
- En cambio matrices dispersas... El costo del shuffle solo aplica a los valores no nulos (que son $\ll n^3$)

Multiplicación de matrices

Casos de uso:

- Matriz de Usuarios x Productos → similitud de usuarios dado productos comprados
- Patrones en grafos
- Cálculo de PageRank, multiplicando por vector $\mathbf{r}$ en vez de otra matriz

$$r_{t+1} = d \cdot M \cdot r_t + \frac{1-d}{N} \mathbf{1}$$

Otras aplicaciones





Patrones en grafos: spam

- Si es que no existe factor de amortiguación en PageRank, podemos tener subgrafos que acumulan importancia artificial: ***rank sink***
- Detectar patrones en red nos puede ayudar en la detección de spam
- Ej: Si un conjunto de papers se citan entre ellos, podríamos sospechar algo raro... $A \rightarrow B \rightarrow C \rightarrow A$
O al menos curioso

TF-IDF en grandes volúmenes

- **Idea:** Calcular relevancia de palabras en grandes colecciones de documentos de forma distribuida
- Contar frecuencias en paralelo (TF)
- Agregar documentos globalmente (DF)

$$w_{x,y} = \text{tf}_{x,y} \times \log \left(\frac{N}{\text{df}_x} \right)$$

TF-IDF

Term x within document y

$\text{tf}_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

Cierre



Cierre

- Con estos ejemplos la idea fue mostrar un poco de que la gracia no está solo en el framework, sino en cómo pensamos los problemas
- Los ejemplos se pueden llevar a muchos contextos. Hay que darles una vuelta para bajarlos a tu necesidad
- En la actividad propuesta veremos otro ejemplo de cómo llevar PageRank fuera de la web

Lectura complementaria

- Paper Map-Reduce:
<https://static.googleusercontent.com/media/research.google.com/es//archive/mapreduce-osdi04.pdf>
- Libro Mining of Massive Datasets, capítulo 2: MapReduce and the new software stack
- Operadores PySpark:
<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.RDD.html>